

Gebze Technical University
Computer Engineering Department

CSE 396 Computer Engineering Project
Module Documentation Report

**MOD-01: Embedded & Hardware
Module**

Autonomous First Responder Fire & Rescue Robot

Name	Student ID	Role
Nuri Ziya Kırtepe	210104004027	Primary - Power & UART
Ömer Nacar	210104004814	Secondary - Motors & Stuck Detection
Gabil Rahimli	230104004902	Secondary - Env. Sensors

Instructor: Salih Sarp
Submission Date: April 2026

Contents

1	Module Overview	2
2	Key Responsibilities	2
3	Internal Responsibilities and Interface Design	2
3.1	Power Management (pwr_management.h)	2
3.2	UART Communication (uart_comm.h)	2
3.3	Motor and Stuck Logic (motor_control.h & stuck_detection.h)	2
3.4	Environmental Drivers (environment_sensors.h)	3
4	Hardware Architecture and Physical Implementation	3
5	Software Interface Summary (API)	3
6	Input / Output Summary	3
7	Known Risks and Mitigations	3
8	Conclusion	4

1 Module Overview

The Embedded and Hardware Control Module (MOD-01) serves as the "reflexive core" of the search and rescue robot. It is responsible for low-level hardware abstraction, real-time motor control, environmental sensing, and safety failsafes. Built on the STM32F103C8T6 (Blue Pill) microcontroller, this module ensures that high-level commands from the Raspberry Pi 5 are translated into physical motion while simultaneously monitoring the robot's health and surroundings through a structured telemetry stream.

2 Key Responsibilities

The functional scope of MOD-01 includes the following technical domains:

- **4WD Motor Control:** Driving four DC motors via the L298N H-Bridge using high-frequency PWM.
- **Stuck Detection:** Real-time IMU-based monitoring to detect and recover from mechanical obstructions.
- **Environmental Sensing:** Polling DHT22 (Temp/Humidity) and MQ-2 (Smoke/-Gas) sensors.
- **Power Management:** Managing a dual-powerbank setup with a 12V decoy trigger and time-based operation limits (60-minute RTH limit).
- **Telemetry Hub:** Aggregating sensor data into a 50Hz structured UART stream for the Raspberry Pi 5.

3 Internal Responsibilities and Interface Design

3.1 Power Management (`pwr_management.h`)

This sub-module handles the 12V Type-C Decoy trigger via `GPIOA_PIN_5`. It monitors the operational duration to enforce a strict safety limit, ensuring the robot initiates Return-to-Home (RTH) protocols before battery depletion.

3.2 UART Communication (`uart_comm.h`)

A 115200 baud UART bridge is maintained for bidirectional communication. MOD-01 serializes environmental data, IMU yaw angles, and ultrasonic distances into a packet structure transmitted at a 20ms period.

3.3 Motor and Stuck Logic (`motor_control.h` & `stuck_detection.h`)

Chassis movement is governed by mapping UART directions (Forward, Backward, Left, Right) to L298N GPIO states. Parallel to this, the MPU6050 IMU is polled via I2C to verify motion. If the motors are active but the accelerometer/gyroscope magnitude falls below defined thresholds (50mg / 5 deg/s) for over 2 seconds, a recovery sequence is triggered.

3.4 Environmental Drivers (environment_sensors.h)

High-level drivers facilitate data acquisition from the DHT22 and MQ-2 sensors. The MQ-2 analog signal is processed through the STM32 ADC, utilizing an internal threshold logic to flag smoke alerts.

4 Hardware Architecture and Physical Implementation

The physical layer follows a strict architectural scheme designed for isolation and safety:

- **Dual Power Isolation:** To prevent brownouts caused by motor stall currents, a dedicated 10000mAh PD powerbank is used for the 4WD system, while a separate supply powers the Pi 5 and STM32.
- **Common Ground Policy:** All power and logic grounds are interconnected to establish a stable reference for UART and I2C communications.
- **Voltage Protection:** 1k/2k Ohm voltage dividers are implemented for non-5V-tolerant pins (e.g., MQ-2 analog out and Ultrasonic Echo) to protect the 3.3V STM32 logic.
- **Pull-up Configuration:** A 4.7k Ohm resistor is utilized between VCC and the Data line of the DHT sensor to ensure signal integrity.

5 Software Interface Summary (API)

The following table summarizes the primary public functions exposed by the module:

Function	Description	Header
<code>pwr_init()</code>	Configures power GPIOs and failsafe timers.	<code>pwr_management.h</code>
<code>uart_send_telemetry()</code>	Packages and transmits sensor data via DMA.	<code>uart_comm.h</code>
<code>motor_set_state()</code>	Sets speed and direction for the 4WD chassis.	<code>motor_control.h</code>
<code>stuck_check()</code>	Validates motion using IMU feedback.	<code>stuck_detection.h</code>
<code>env_read_all()</code>	Reads all environmental telemetry in one call.	<code>environment_sensors.h</code>

6 Input / Output Summary

Type	Source/Target	Description
Input	Sensors	ADC (MQ-2), I2C (IMU), GPIO (Ultrasonic/DHT).
Input	MOD-04 (Pi 5)	Direction and Speed commands via UART.
Output	L298N Driver	PWM duty cycles and H-Bridge logic pins.
Output	MOD-04 (Pi 5)	Aggregated telemetry JSON/Struct over UART.
Output	Indicators	LED and Buzzer status feedback.

7 Known Risks and Mitigations

- **Motor Drift:** Mechanical differences in motors cause the robot to veer. *Mitigation:* Implementation of `motor_set_trim()` to balance PWM signals.

- **IMU Gyro Drift:** Yaw angle accumulates error over time. *Mitigation:* Future integration of accelerometer-based tilt compensation.
- **UART Packet Loss:** Electrical noise may corrupt data. *Mitigation:* Use of twisted-pair wiring and future software-side checksums.

8 Conclusion

MOD-01 provides the essential physical foundation for the search and rescue robot, ensuring that high-level autonomous intent is executed reliably. By centralizing real-time motor control, environmental sensing, and hardware-level failsafes into the STM32 firmware, the system maintains a robust response to both internal faults and external hazards, laying the groundwork for successful mission deployment.